

AI Business Intelligence Agent - System Analysis & Design

1. Problem Statement & Objectives

Modern organizations struggle to extract actionable, strategic insights from massive volumes of disconnected raw data across sales, marketing, and operations. Traditional dashboards require manual query formulation and rely heavily on human interpretation, which is prone to error and delay. The objective of this project is to develop an autonomous AI Business Intelligence (BI) Agent that acts as a virtual consultant. The system aims to automate the identification of trends, predict future inventory and sales via time-series modeling, and utilize a RAG-powered Large Language Model (LLM) to generate context-aware, strategic business decisions.

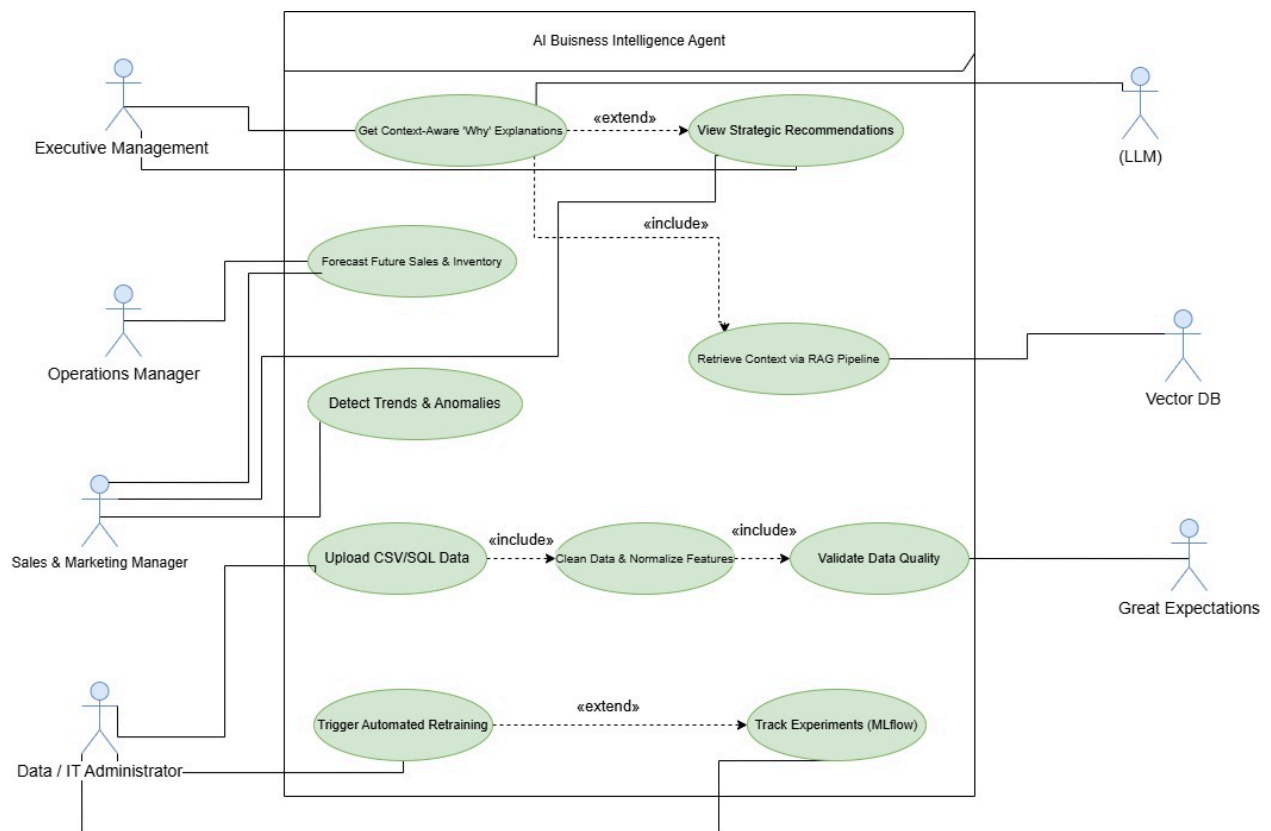
2. Use Case Diagram & Descriptions

2.1 System Actors & Interactions

The system is designed to interact with the following primary actors:

- **Executive Management:** Interacts with the system to consume high-level strategic recommendations and automated insights across the unified dashboard.
- **Sales & Marketing Managers:** Utilizes the system to view predictive forecasting for future sales and receives context-aware recommendations for marketing promotions.
- **Operations & Supply Chain Managers:** Accesses inventory requirement forecasts and acts upon automated suggestions to adjust stock levels.
- **Data/IT Administrators:** Responsible for maintaining the data ingestion pipeline, uploading raw datasets (CSV/SQL), and monitoring the MLflow tracking environment.

2.2 Use Case Diagram



3. Functional & Non-Functional Requirements

3.1 Functional Requirements

- **Data Ingestion:** The system must provide secure API endpoints for uploading CSV and SQL data.
- **Automated Preprocessing:** The system must automatically clean uploaded data, handle missing values, and normalize features.
- **Predictive Forecasting:** The system must deploy time-series machine learning models (e.g., XGBoost, Prophet) for demand and sales forecasting.
- **RAG Pipeline:** The system must utilize a Vector Database to store business data embeddings and retrieve context for the LLM.
- **Decision-Making Engine:** The system must generate context-aware, text-based recommendations using an Enterprise LLM API.
- **Dashboard Access:** The system must provide a live, unified dashboard for real-time interaction.

3.2 Non-Functional Requirements

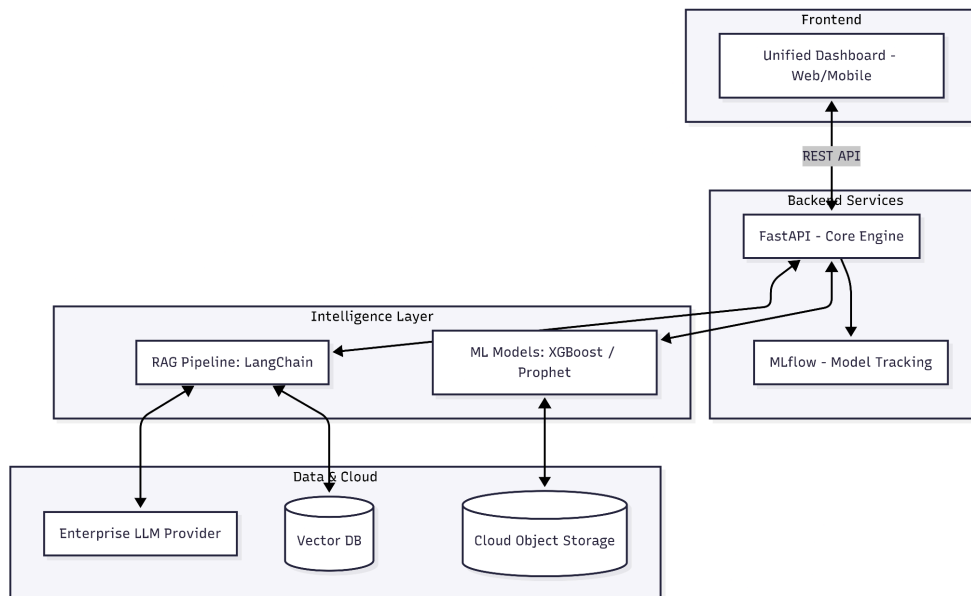
- **Usability (Cross-Platform):** The interface must ensure a seamless experience across Web and Mobile platforms.
- **Scalability:** Backend services must be containerized to scale dynamically based on data load.
- **Security:** The data ingestion pipeline must encrypt data in transit and restrict endpoint access via authentication.
- **Reliability:** Automated retraining pipelines must be triggered automatically upon detecting data drift to maintain prediction accuracy.

4. Software Architecture

The system utilizes a **Microservices-oriented Architecture** to ensure scalability and clear separation of concerns between data processing, machine learning, and user interface layers.

- **Presentation Layer (Frontend):** A unified, cross-platform dashboard built for real-time interaction.
- **Application Layer (Backend):** Powered by Python and FastAPI, this layer exposes RESTful endpoints, handles data ingestion, and orchestrates the AI logic.
- **Intelligence Layer (ML & AI):** Houses the machine learning models registered via MLflow, and the RAG pipeline utilizing LangChain and an Enterprise LLM Provider (e.g., Azure OpenAI, AWS Bedrock).
- **Data Layer:** Comprises raw data storage via Cloud Object Storage (e.g., Azure Blob, Amazon S3), analytical data schemas, and the Vector Database for embeddings.

4.1 Architecture Diagram



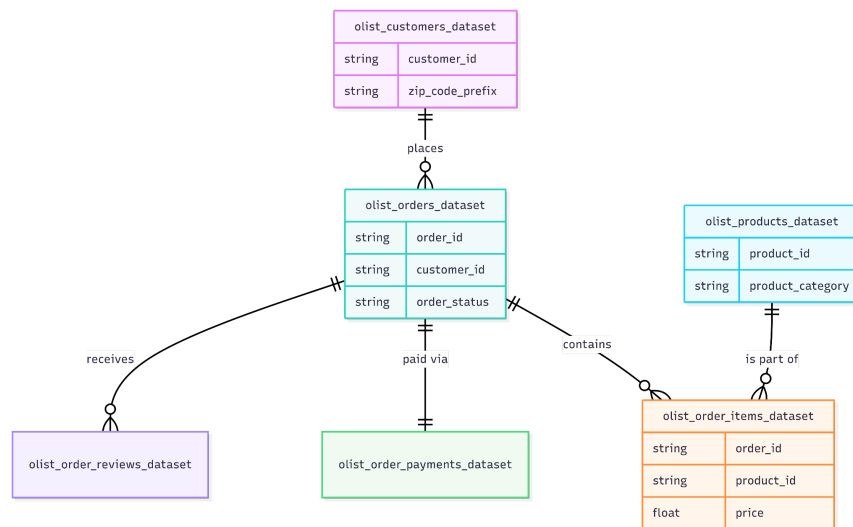
5. Database Design & Data Modeling

While the system is primarily an analytical engine rather than a traditional transactional application, robust database design is critical. The architecture employs a dual-database strategy:

5.1 Relational/Analytical Schema (The E-Commerce Dataset)

The system ingests structured relational data. The ER Diagram for this module reflects a central Fact table surrounded by Dimension tables (Star/Snowflake Schema).

- **olist_orders_dataset (Fact):** The central entity storing order IDs, timestamps, and status.
- **olist_order_items_dataset (Dimension):** Links orders to specific products and prices.
- **olist_customers_dataset (Dimension):** Contains customer demographics and geolocation references.
- **olist_products_dataset (Dimension):** Catalogs product categories and specifications.



5.2 Vector Database

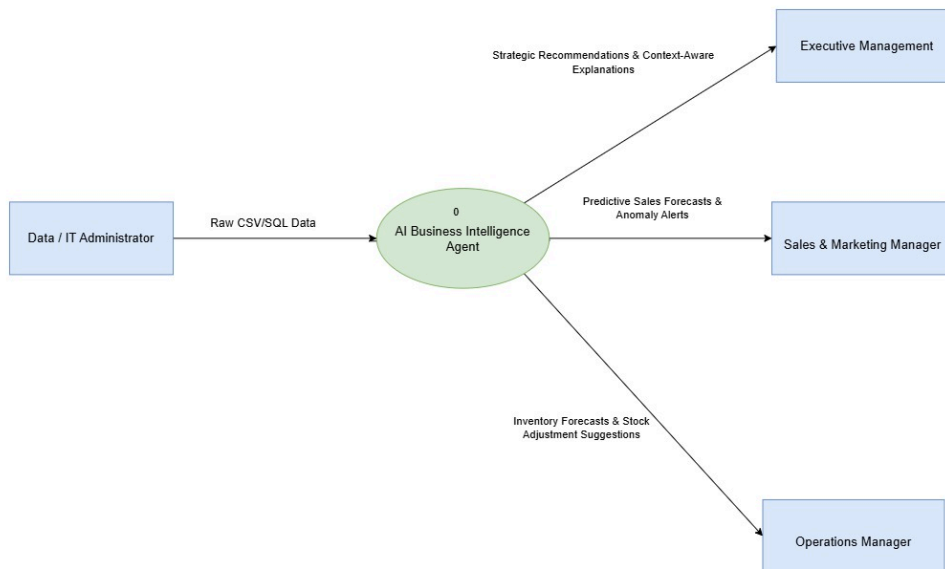
To enable the RAG pipeline, a Managed Vector Database (e.g., Azure AI Search, Pinecone, or ChromaDB) is utilized.

- **Logical Schema:** Stores document chunks (e.g., text summaries of sales trends, anomaly alerts) alongside their high-dimensional vector embeddings and metadata tags.

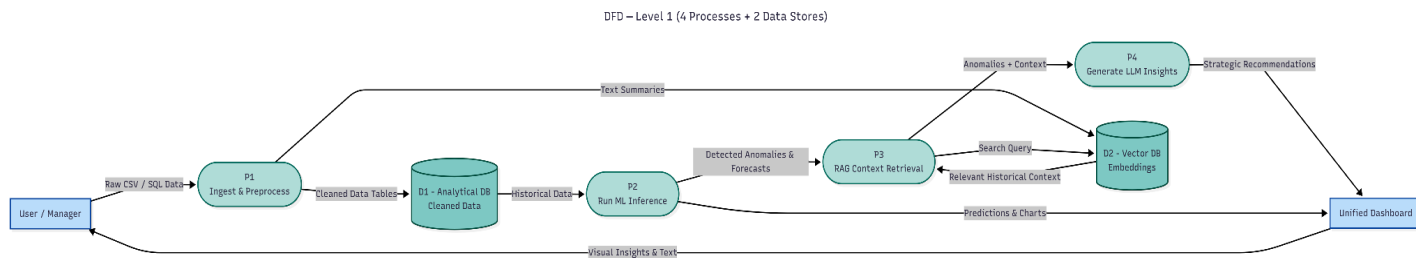
6. Data Flow & System Behavior

6.1 DFD (Data Flow Diagram)

- **Level 0 (Context):** The User (Manager) inputs raw CSV/SQL data into the System. The System processes this data and outputs Predictive Forecasts and AI Recommendations back to the User.

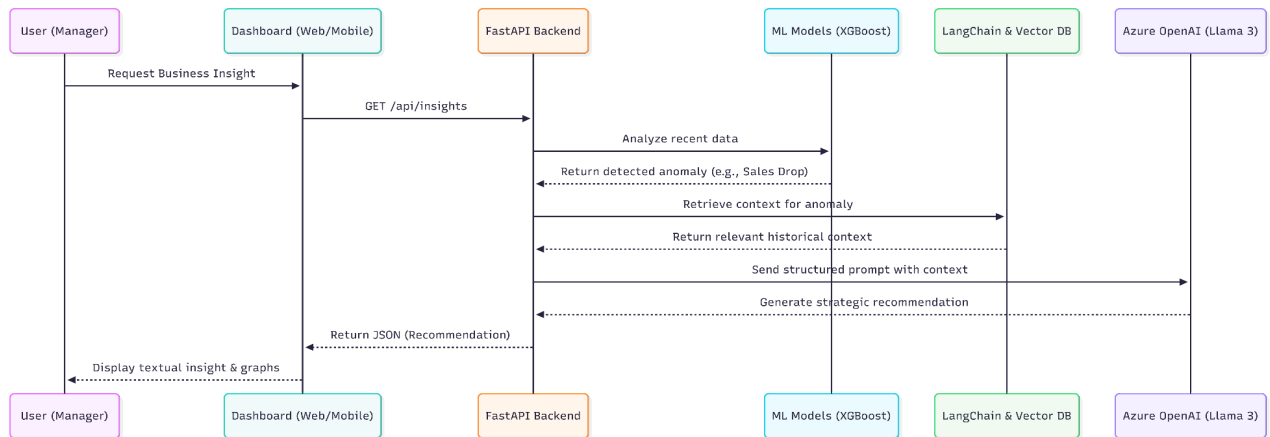


- **Level 1:** Process 1 (Ingestion) validates data. Process 2 (ML Pipeline) detects anomalies. Process 3 (RAG) retrieves context. Process 4 (LLM) generates the final text insight.

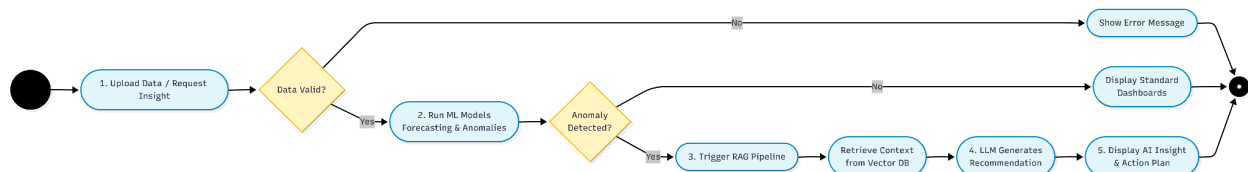


6.2 Sequence Diagram

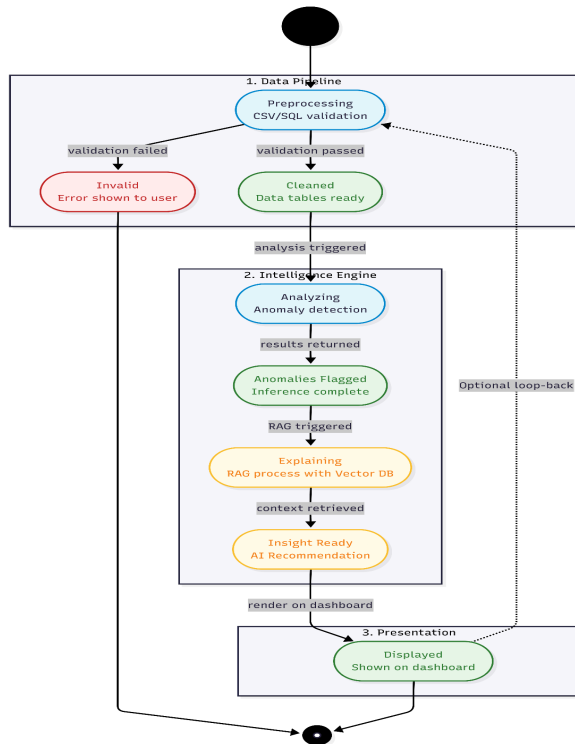
1. User requests an insight via the Dashboard.
2. Dashboard calls the FastAPI backend.
3. FastAPI triggers the Anomaly Detection Model.
4. Model returns a flagged trend (e.g., "Sales Drop in Region X").
5. FastAPI sends the trend to LangChain.
6. LangChain queries the Vector DB for context.
7. Vector DB returns relevant historical data.
8. LangChain formats the Prompt and sends it to the LLM.
9. LLM returns the strategic recommendation.
10. FastAPI routes the response back to the Dashboard.



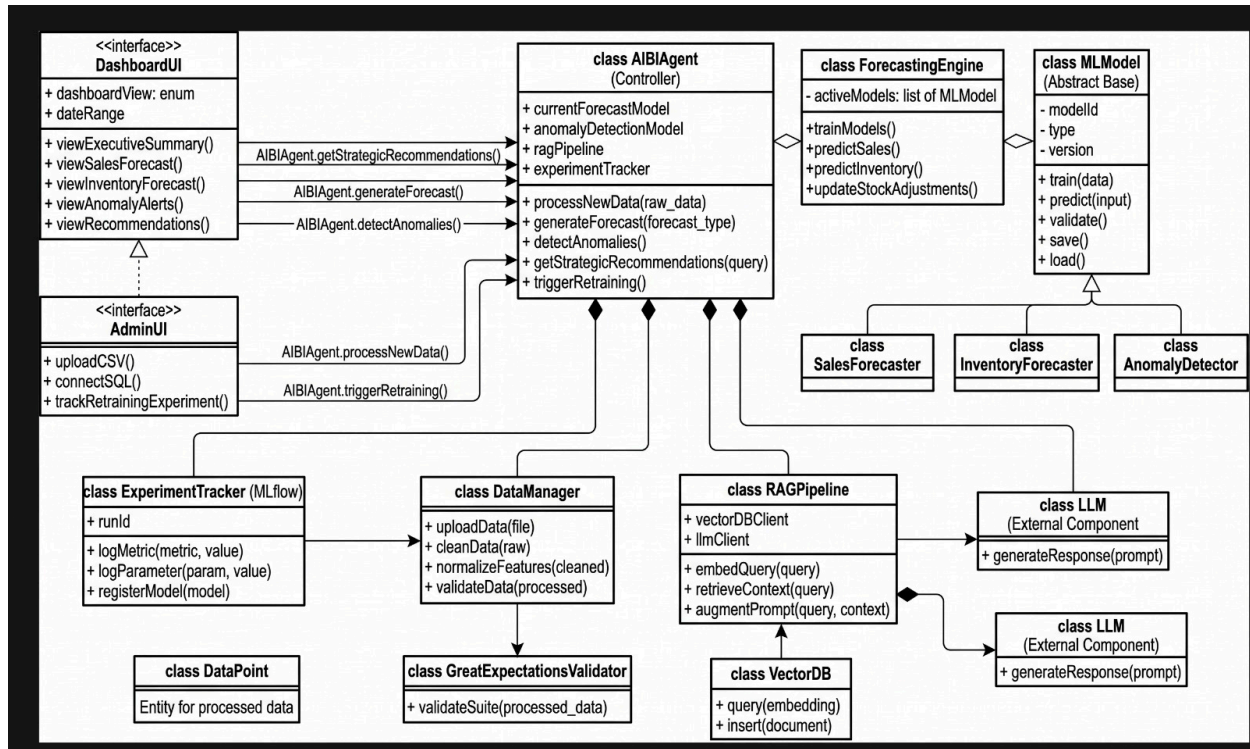
6.3 Activity Diagram



6.4 State Diagram



6.4 Class Diagram



7. UI/UX Design & Prototyping

7.1 UI/UX Guidelines

The application interface will adopt a high-end, professional aesthetic, aligning with premium technology brand standards (e.g., the TechIavelli identity). The design principles are focused on data readability and executive presentation.

- **Color Palette:** The primary interface utilizes a sophisticated dark theme (deep charcoal or black backgrounds) to reduce eye strain during prolonged analytical sessions. This is accented with elegant gold and navy blue highlights to draw attention to critical AI recommendations, anomalies, and primary action buttons.
- **Typography:** Clean, modern sans-serif fonts (e.g., Roboto or Inter) for maximum legibility of data tables and executive summaries.
- **Layout:** A modular, card-based dashboard layout allowing users to view real-time charts alongside the AI's textual reasoning without overwhelming the screen.

8. System Deployment & Integration

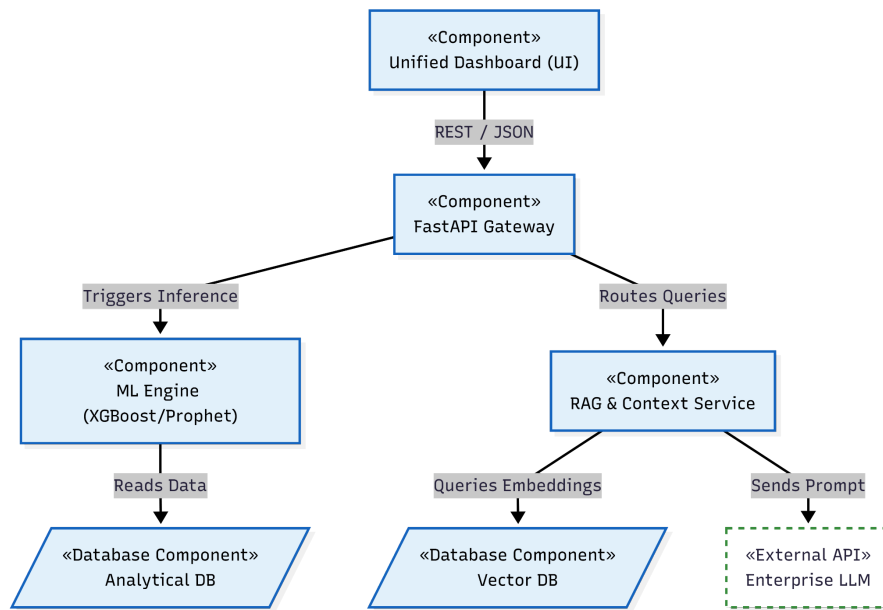
8.1 Technology Stack Summary

Component	Technology
Frontend	Cross-platform framework (Web/Mobile)
Backend API	Python, FastAPI
Machine Learning	Scikit-learn, XGBoost, Prophet
Generative AI	LangChain, Vector DB, Enterprise LLM
MLOps & DevOps	MLflow, Docker

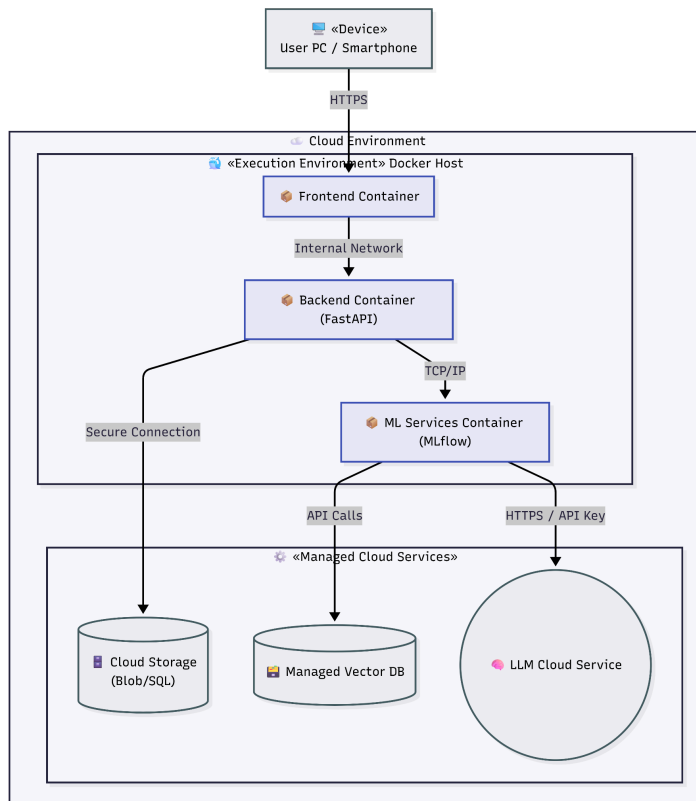
8.2 Deployment Strategy

To ensure a smooth deployment process, the system relies on containerization. The backend services (FastAPI, MLflow, Vector DB) will be packaged using Docker. The deployment strategy focuses on automated pipelines and GUI-based management tools (such as Docker Desktop) to streamline operations, avoiding the necessity of complex manual terminal command executions.

8.3 Component Diagram



8.4 Deployment Diagram



WireFrame :

